LLM Module Group Assignment



Power Agent

An LLM Agent for Estimating and Reducing Household Electricity Bills

Technical Report

Members:

J.G.D. Nihansa Damdini | CClarke589

S.Vishmi Dewmini De Silva | CClarke529

Repository: https://github.com/Nihansadamdini/power-agent

Table of Contents

1. Problem and Scenario.....3

2. System Architecture.....3

3. Agent.....4

4. Tools.....4

5. Agent Loop.....5

6. Prompt Design.....6

7. Testing.....6

8. Challenges.....8

9. Limitations.....8

10. Future Improvements.....9

11. Individual Contributions.....9

1. Problem and Scenario

Household electricity in Sri Lanka is billed using a tiered system called a block tariff. Each extra unit of electricity (measured in kWh) costs more than the one before it. A fixed monthly charge also applies, and it changes based on how much you use. Because the pricing is not straightforward and jumps between slabs, most people cannot easily guess how changing their appliance use will affect their bill.

Power Agent is a command-line helper. You can tell it about your appliances in plain English, for example “Add a 1200W AC used 6 hours a day.” It then calculates your electricity use and estimated bill. It also lets you run “what if” tests, for example: what if you turn off the AC for two hours a day? It can also save a custom savings plan as a markdown file.

The main goal is for a household user to ask, “Which appliance should I cut to save the most? Save a plan.” This is not a single-step question. It needs several steps:

It needs a steps:

- First read the list of appliances you’ve already added.
- Then simulate turning off each one one by one.
- Compare how much each change saves.
- Finally write a plan showing which cut brings the biggest saving.

That’s why Power Agent is designed as a tool-using agent, not a regular prompt. The model has to pick the tool to run, wait for results and then move to the next step. It’s like having an assistant that thinks step by step instead of just giving a one-time answer.

2. System Architecture

The system has six parts:

- **main.py**: the command-line interface. It validates input (non-empty, at most 1,000 characters) and supports **reset** and **exit**
- **agent/**: the agent loop **(loop.py)** and logging **(logger.py)**
- **tools/**: the five tool functions and their JSON schemas.
- **tariff.json**: the tariff configuration (block rates and fixed charges).
- **data/appliances.json**: persistent appliance storage, excluded from Git.
- **plans/**: saved savings plans as markdown files.

The language model runs locally through Ollama. The default is **qwen2.5:7b**,and it can be changed with the **AGENT_MODEL** environment variable. Running locally keeps household data on the user's machine and avoids API costs.

Request flow: the user types a message, and **main.py** validates it. **chat()** sends the conversation history and the tool schemas to the model. If the model asks to call tools, the loop

runs them through the **TOOL_FUNCTIONS** registry and appends the results to the history. The loop repeats until the model answers in plain text, and that answer is printed.

Key design decision: the agent and the tools meet only at a fixed contract, **TOOL_FUNCTIONS** (name to function) and **TOOL_SCHEMAS** (JSON schemas). This lets two team members work on separate Git branches in parallel without editing the same files.

3. Agent

Power Agent uses a single tool-calling agent. One model receives the system prompt, the conversation and the tool schemas, and decides at each step whether to call a tool or answer. A single agent was enough because every task uses the same small toolset and the steps depend on one another. Splitting it into several agents would add coordination cost without a clear benefit. The agent's role is fixed by its system prompt, which casts it as a household electricity bill optimizer for Sri Lankan homes using the CEB domestic tariff. Two limits keep it predictable: at most 8 rounds of tool calls per request, and a history of at most 20 messages after the system prompt.

4. Tools

Tool	Purpose	Inputs	Returns
add_appliance	Add or update an appliance	name, watts, hours_per_day, days_per_month (default 30)	saved name, monthly kWh
list_appliances	List all appliances	none	items with monthly kWh, total units
calculate_bill	Monthly bill for a number of units	units	units, energy charge, fixed charge, total, tariff note
simulate_change	What-if on one appliance's daily hours	appliance, new_hours_per_day	units and bill before and after, monthly saving
save_plan	Save a markdown savings plan	title, content	saved file name

Billing logic.

For each tariff block, the units that fall inside that block are multiplied by the block's rate, and the results are added. The fixed charge is then chosen from the band that contains the total units.**simulate_change** computes the total units before the change, replaces the chosen appliance's hours, recomputes the units, calculates both bills and returns the difference.

Input validation.

Every tool rejects bad input with a clear **ValueError:** name 1-50 characters, watts 1-10,000, hours per day 0-24, days per month 1-31, units 0-5,000, and plan content up to 5,000 characters. The agent loop turns the exception into an {“error”:”....”} result, so the model can read the problem and correct itself instead of crashing the program.

Safe file handling.

save_plan builds the file name from a sanitised slug (letters, digits and hyphens, at most 40 characters) plus a timestamp. The file is always written to the fixed plans/ folder, so user text can never become part of a file path.

Type coercion.

Small models often send numbers as strings (the trace showed **“4”** for hours). All tools convert inputs with **float()** before validating, so this does not cause failures.

5. Agent Loop

The loop in **agent/loop.py** follows the standard tool-calling pattern:

1. Append the user's message to the history.
2. Call **ollama.chat** with the history and **TOOL_SCHEMAS.**
3. If the reply contains tool calls, run each one through a safe wrapper. The wrapper looks the function up in **TOOL_FUNCTIONS**, catches exceptions, and returns either the result or an error dictionary.
4. Append each tool result to the history and go back to step 2.
5. When the model replies without tool calls, return that text as the final answer.

Every call is printed as a trace, for example **[step 1] -> simulate_change({...})** followed by **<-{...},** which is shown in the demo video. **trim_history** keeps the system message and the last 20 messages, always starting from a user message so a tool result is never separated from its tool call. If Ollama cannot be reached, a clear message tells the user to start **ollama serve** and check that the model is pulled. The loop stops after 8 rounds of tool calls per request (**MAX_STEPS**), so a confused model cannot loop forever. If the limit is reached, the user is asked to simplify the request. Unknown tool names, arguments that are not a JSON object, and unexpected tool crashes are all returned to the model as error results instead of stopping the program, and each step (tool, arguments, result and time in milliseconds) is also written to the log.

6. Prompt Design

Prompting works on two levels.

System prompt.

The prompt gives the agent a role and a short list of rules. It must always use tools for appliance data, units and bill amounts, and never guess or invent numbers. It also spells out the workflow for finding savings: call **list_appliances**, then **calculate_bill** with the total units, then **simulate_change** on the biggest consumers. If the user asks to save a plan, it calls **save_plan** with a clear markdown summary. If a tool returns an error, it must read the message, fix the arguments and retry, or ask the user for the missing information. Amounts are in rupees (Rs.), units are kWh, and the final answer is a short summary in the language the user wrote in. Putting the workflow in the prompt, instead of hard-coding it, keeps the loop generic and lets the model adapt the steps to the request.

Tool descriptions and schemas.

Because the model chooses tools from these descriptions, they are written to be short and action-first. Parameter names are consistent across tools (**hours_per_day**, **new_hours_per_day**) and carry units (watts, kWh). Only the necessary parameters are marked required, and **days_per_month** defaults to 30 so the model does not need to ask for it. The **calculate_bill** description states that rates come from **tariff.json** and may be sample values. The tool also returns a **tariff_note** field, so the model can pass this caveat to the user.

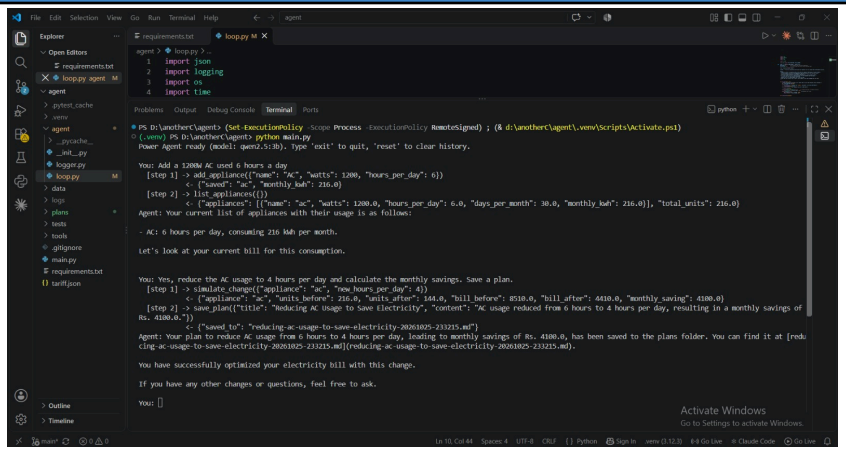
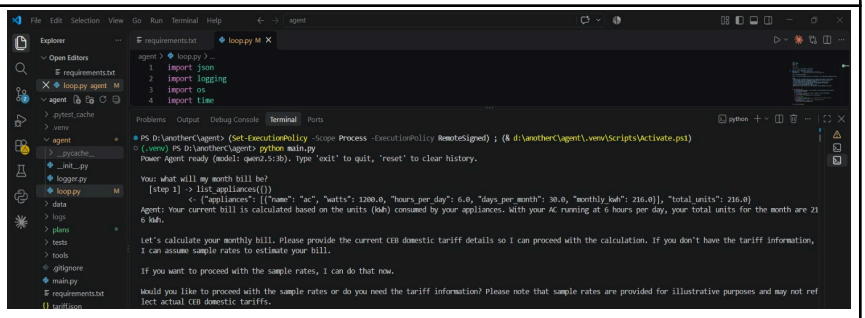
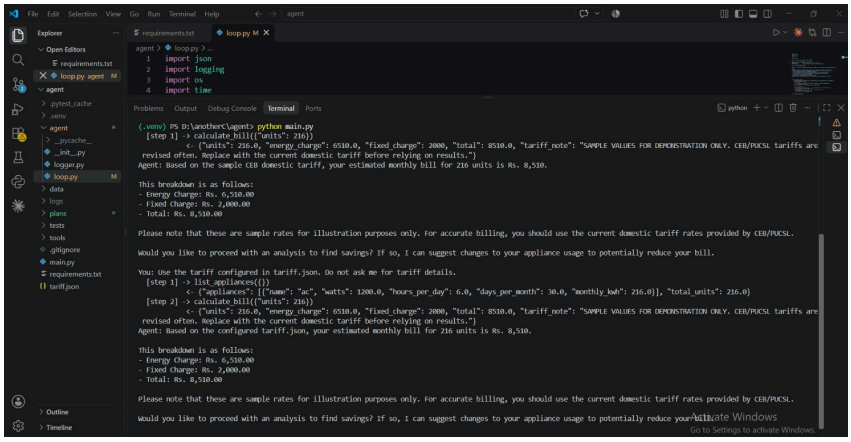
7. Testing

Unit tests.

Nine pytest tests cover the tools: zero units gives no energy charge; the bill rises with units; negative units are rejected; invalid watts and hours are rejected; the tariff note is returned; adding and listing an appliance gives the correct total (1200 W x 6 h x 30 days = 216 kWh); simulating a reduction saves money; and an unknown appliance is rejected. A fixture redirects the appliance database to a temporary file, so tests never touch real data. Result: **9 passed**.

Agent-level tests.

The prompts below were run against the full agent.

Prompt	Expected behaviour	Result
Add a 1200W AC used 6 hours a day	**add_appliance** called	
What will my monthly bill be?	**list_appliances**, then **calculate_bill**	
What if I use the AC only 4 hours a day?	**simulate_change**	

Observed error handling.

When the what-if question was asked before any appliance existed, the trace showed the error **Unknown appliance 'ac'. Known: []**. The input **AC** had also been normalised to **ac**. The error went back to the model as data, and the program continued normally.

8. Challenges

- **Test imports.** pytest first failed with **No module named 'tools'**. Adding an empty **tests/__init__.py** fixed it.
- **Local model environment.** On the development laptop (8 GB RAM, with about 1.5 GB free), the model produced corrupted output and the machine often froze. We forced CPU mode (**OLLAMA_LLM_LIBRARY=cpu**, **CUDA_VISIBLE_DEVICES=-1**), closed other applications and tried the smaller models already installed. A larger model could not be downloaded because of a limited data allowance. The final demo was run with **qwen2.5:3b** model on the same 8 GB RAM laptop in CPU mode.
- **Small-model reliability.** The 1B model is too small to call tools reliably. Smaller models can skip a tool call or choose the wrong one, so tool descriptions needed careful wording.
- **Changing tariffs.** CEB and PUCSL have revised domestic rates several times, so rates were moved out of the code into **tariff.json**.
- **Team workflow.** Each member worked on a separate branch and merged by pull request, with small commits per feature. This avoided file conflicts.

9. Limitations

- The bundled tariff values are **sample data**, not the official current CEB/PUCSL rates. Bill estimates are illustrative until **tariff.json** is updated.
- Estimates assume constant daily usage and ignore standby power, seasonal changes, time-of-use pricing and solar net metering.
- There is one shared appliance store and no user accounts, so it supports a single household.
- Local small models can make mistakes with tool choice and arithmetic summaries, and are slow on low-memory hardware.
- Conversation history exists only for the current session, and each message is limited to 1,000 characters.

10. Future Improvements

- Version the tariff with effective dates and load the official rates automatically from PUCSL or CEB.
- Support other consumer categories and multiple households or users.
- Add an automated evaluation set that scores the agent's tool choices.
- Offer a hosted model option as a fallback when local hardware is too weak.
- Ask for confirmation before saving a plan, and add charts of savings.
- Add appliance presets (typical wattage for common appliances) so users do not need to know the rating.

11. Individual Contributions

Full Name	Index Number	Contribution
J.G.D. Nihansa Damdini	CClarke589	Developed the core AI agent functionality that connects the local language model with the application's tools, enabling natural-language interaction, automated tool execution, and multi-step electricity usage analysis.
S.Vishmi Dewmini De Silva	CClarke529	Built the five agent tools with input validation, the tool schemas, tariff.json, and the nine unit tests for the tools.